💧 Prepare to Commit (Preparar para salvar mudanças)

Essa parte do seu PDF é onde o Git **deixa de ser teoria e vira ferramenta real**.

🧠 Primeiro: entenda o fluxo mental

Antes dos comandos, fixa isso:

```
Você edita → você escolhe o que salvar → você salva
```

No Git isso vira:

```
Working → Staging → Commit
```

👉 Aqui está o segredo do Git.

📦 1. `git status` — O "radar" do projeto

📌 Comando:

```
git status
```

🧠 O que ele faz?

Mostra:

- arquivos modificados
- arquivos novos
- arquivos prontos para commit

🔧 Exemplo prático

Você cria um arquivo:

```
echo "Olá" > app.txt
git status
```

Saída (simplificada):

```
untracked files:
  app.txt
```

👉 Tradução:

“Esse arquivo existe, mas o Git ainda não está controlando ele”

💡 Use sempre

`git status` é seu melhor amigo no Git

Use antes de qualquer ação.

➕ 2. `git add` — Preparando o que será salvo

📌 Comando:

```
git add arquivo.txt
```

ou tudo:

```
git add .
```

🧠 O que ele faz?

Move arquivos para a **Staging Area** (área de preparação)

👉 É como dizer:

“Quero incluir isso no próximo salvamento”

🔧 Exemplo

```
git add app.txt
git status
```

Agora aparece:

```
changes to be committed:
  app.txt
```

🎯 Analogia

- Working = rascunho
- Staging = itens selecionados
- Commit = versão final salva

⚠️ Erro comum

👉 Usar `git add .` sem pensar

Isso pode adicionar:

- arquivos desnecessários
- lixo

- arquivos sensíveis

🛠️ 3. `git add -p` — Controle avançado

📌 Comando:

```
git add -p
```

🧠 O que faz?

Permite escolher **parte do código** para adicionar

👉 Muito usado por programadores avançados

🔧 Exemplo real

Você editou um arquivo com:

- correção de bug
- nova funcionalidade

Com `git add -p`, você separa isso em commits diferentes.

💡 Mentalidade de engenharia

Um commit = uma ideia

🔄 4. `git mv` — Renomear arquivo

```
git mv antigo.txt novo.txt
```

👉 Git entende que foi uma mudança, não um arquivo novo

❌ 5. `git rm` — Remover arquivo

```
git rm arquivo.txt
```

👉 Remove e já prepara para commit

💧 Versão importante:

```
git rm --cached arquivo.txt
```

👉 Para de rastrear, mas NÃO apaga do PC

💡 Uso prático

Muito usado com `.gitignore`:

- remover arquivos do controle (ex: senha, logs)

↩️ 6. `git reset` — Tirar do staging

📌 Comando:

```
git reset arquivo.txt
```

ou tudo:

```
git reset
```

🧠 O que faz?

Remove arquivos da staging area

👉 Mas NÃO apaga o que você fez

🔧 Exemplo

Você fez:

```
git add .
```

Mas adicionou coisa errada.

Então:

```
git reset
```

👉 Corrige sem perder nada

🔍 7. Resumo visual completo

```
1. Você edita arquivos
   ↓
2. git status (ver situação)
   ↓
3. git add (escolher o que salvar)
   ↓
4. git commit (salvar versão)
```

🔥 Exemplo completo (vida real)

```
git init

echo "linha 1" > app.txt
git status

git add app.txt
git commit -m "primeira versão"

echo "linha 2" >> app.txt

git status
git add .
git commit -m "adiciona nova linha"
```

🧠 O que você aprendeu aqui (nível engenharia)

Você não está só usando comandos.

Você está:

- controlando versões
- separando mudanças
- organizando histórico
- pensando como desenvolvedor profissional

🚀 Próximo nível (muito importante)

Agora vem a parte onde o Git ganha poder de verdade:

👉 **Make Commits (git commit avançado + boas práticas)**